

Doctor Visit Booking API

Technical Documentation

REST API for managing doctor visit bookings

FastAPI | SQLAlchemy 2.0 | SQLite | JWT Auth

Version 0.1.0

Table of Contents

- 1. API Usage Guide
 - 1.1 Authentication
 - 1.2 Doctors
 - 1.3 Patients
 - 1.4 Working Hours & Schedules
 - 1.5 Appointments
 - 1.6 Error Codes Reference
- 2. Architecture Description
 - 2.1 High-Level Architecture
 - 2.2 Project Structure
 - 2.3 Database Schema (ER Diagram)
 - 2.4 Class Diagram
 - 2.5 Sequence Diagrams
- 3. Self-Analysis
 - 3.1 SOLID Principles Adherence
 - 3.2 Design Trade-offs
 - 3.3 Areas for Improvement

1. API Usage Guide

The API is served at <http://localhost:8000> by default. Interactive Swagger UI is available at </docs> and ReDoc at </redoc>. All request and response bodies use JSON. Protected endpoints require an Authorization: Bearer <token> header.

1.1 Authentication

The system uses JWT (JSON Web Tokens) for authentication. Tokens are obtained via the login endpoint and expire after 60 minutes (configurable). Doctors and patients register separately; both receive a token on successful registration.

POST /auth/register/doctor

Register a new doctor account with initial working hours.

Request:

```
POST /auth/register/doctor
Content-Type: application/json

{
  "name": "Dr. Smith",
  "email": "smith@clinic.com",
  "password": "securepass123",
  "address": "123 Medical Ave",
  "working_hours": [
    {
      "day_of_week": 0,
      "start_time": "09:00:00",
      "end_time": "17:00:00",
      "is_break": false
    }
  ]
}
```

Response (201 Created):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...\"",
  "token_type": "bearer"
}
```

POST /auth/register/patient

Register a new patient account linked to a personal doctor.

Request:

```
POST /auth/register/patient
Content-Type: application/json

{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "securepass123",
}
```

```
"phone": "+359888000000",
"doctor_id": 1
}
```

Response (201 Created):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

POST /auth/login

Authenticate with email and password.

Request:

```
POST /auth/login
Content-Type: application/json

{
  "email": "smith@clinic.com",
  "password": "securepass123"
}
```

Response (200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

Error (401 Unauthorized):

```
{
  "detail": {
    "code": "INVALID_CREDENTIALS",
    "message": "Incorrect email or password."
  }
}
```

1.2 Doctors

GET /doctors

List all registered doctors (public, paginated).

Query Parameters: skip (default 0), limit (default 20, max 100)

Response (200 OK):

```
{
  "items": [
    {
      "id": 1,
      "name": "Dr. Smith",
      "email": "smith@clinic.com",
      "address": "123 Medical Ave",
      "working_hours": [
        {
          "id": 1,
          "day_of_week": 0,

```

```

        "start_time": "09:00:00",
        "end_time": "17:00:00",
        "is_break": false
    }
]
},
"total": 1,
"skip": 0,
"limit": 20
}

```

GET /doctors/{id}

Get a specific doctor's profile and working hours.

Response (200 OK): Same structure as a single item above.

Error (404 Not Found):

```

{
  "detail": {
    "code": "DOCTOR_NOT_FOUND",
    "message": "No doctor found with the given id."
  }
}

```

1.3 Patients

GET /patients/me

Get the authenticated patient's own profile. Requires patient role.

Headers: Authorization: Bearer <token>

Response (200 OK):

```

{
  "id": 2,
  "name": "Jane Doe",
  "email": "jane@example.com",
  "phone": "+359888000000",
  "doctor_id": 1
}

```

1.4 Working Hours & Schedules

Doctors manage three layers of schedule data. When resolving the effective schedule for a given date, the system checks in order: (1) active temporary override covering the date, (2) latest permanent change with `effective_date` $\leq$ the date, (3) base working hours. The first match wins.

PUT /doctors/me/schedule

Replace the doctor's base working hours (doctor only).

Request:

```
PUT /doctors/me/schedule
Authorization: Bearer <doctor_token>

{
  "slots": [
    {"day_of_week": 0, "start_time": "09:00:00",
     "end_time": "12:00:00", "is_break": false},
    {"day_of_week": 0, "start_time": "12:00:00",
     "end_time": "13:00:00", "is_break": true},
    {"day_of_week": 0, "start_time": "13:00:00",
     "end_time": "17:00:00", "is_break": false}
  ]
}
```

Returns 200 with the updated `WeeklyScheduleResponse`. Returns 422 `SCHEDULE_CONFLICTS_APPOINTMENT` if existing appointments would fall outside the new hours.

GET /doctors/{id}/schedule?date=YYYY-MM-DD

Get the effective schedule for a doctor on a specific date. Defaults to today if date is omitted. Returns a list of `TimeSlotResponse` objects.

POST /doctors/me/schedule/temporary

Create a temporary schedule override (max 1 per doctor).

Request:

```
{
  "start_datetime": "2025-06-01T00:00:00Z",
  "end_datetime": "2025-06-08T00:00:00Z",
  "schedule": [
    {"day_of_week": 0, "start_time": "10:00:00",
     "end_time": "14:00:00", "is_break": false}
  ]
}
```

Returns 201 with `TemporaryOverrideResponse`. Returns 409 `OVERWRITE_EXISTS` if the doctor already has one. Returns 422 `SCHEDULE_CONFLICTS_APPOINTMENT` if existing appointments conflict.

DELETE /doctors/me/schedule/temporary

Remove the active temporary override. Returns 204 No Content.

POST /doctors/me/schedule/permanent

Schedule a permanent working-hours replacement (effective_date >= today + 7 days).

Request:

```
{
  "effective_date": "2025-07-01",
  "schedule": [
    {"day_of_week": 0, "start_time": "08:00:00",
     "end_time": "16:00:00", "is_break": false}
  ]
}
```

Returns 201 with PermanentChangeResponse. Returns 422 EFFECTIVE_DATE_TOO_SOON if the date is less than 7 days away. Returns 422 SCHEDULE_CONFLICTS_APPOINTMENT if existing appointments conflict. Pending permanent changes are automatically promoted to base working_hours on application startup and before each schedule resolution.

1.5 Appointments

POST /appointments

Create an appointment (patient only). Business rules enforced:

- Doctor must be the patient's personal doctor (403 NOT_PERSONAL_DOCTOR)
- Start must be at least 24 hours in the future (422 TOO_SOON)
- Start and end must be on the same calendar day (422 INVALID_TIME_RANGE)
- Interval must fit within a non-break working slot (422 OUTSIDE_WORKING_HOURS)
- No overlap with other scheduled appointments (409 APPOINTMENT_OVERLAP)

Request:

```
POST /appointments
Authorization: Bearer <patient_token>

{
  "doctor_id": 1,
  "start_datetime": "2025-06-15T10:00:00Z",
  "end_datetime": "2025-06-15T11:00:00Z"
}
```

Response (201 Created):

```
{
  "id": 1,
  "doctor_id": 1,
  "patient_id": 2,
  "start_datetime": "2025-06-15T10:00:00+00:00",
  "end_datetime": "2025-06-15T11:00:00+00:00",
  "status": "scheduled",
  "cancelled_by": null
}
```

DELETE /appointments/{id}

Cancel an appointment (patient or doctor). Rules:

- User must be the appointment's doctor or patient (403 NOT_APPOINTMENT_OWNER)
- Cancellation at least 12 hours before start (422 CANCELLATION_TOO_LATE)
- Cannot cancel an already-cancelled appointment (409 ALREADY_CANCELLED)

Returns 204 No Content on success.

GET /appointments

List the authenticated user's own appointments. Doctors see appointments where they are the doctor; patients see theirs.

Query Parameters:

- skip, limit - pagination (defaults: 0, 20)
- date_from, date_to - filter by date range (ISO format)
- status - filter by 'scheduled' or 'cancelled'

Response (200 OK):

```
{
```



```
"items": [ ... ],  
"total": 5,  
"skip": 0,  
"limit": 20  
}
```

1.6 Error Codes Reference

All application errors return a consistent JSON structure:

```
{
  "detail": {
    "code": "ERROR_CODE",
    "message": "Human-readable description."
  }
}
```

HTTP	Code	Description
401	INVALID_CREDENTIALS	Wrong email or password
401	INVALID_TOKEN	Missing, expired, or malformed JWT
403	NOT_PERSONAL_DOCTOR	Doctor is not the patient's assigned doctor
403	NOT_APPOINTMENT_OWNER	User is not party to the appointment
403	DOCTOR_REQUIRED	Endpoint requires doctor role
403	PATIENT_REQUIRED	Endpoint requires patient role
404	DOCTOR_NOT_FOUND	No doctor with the given ID
404	PATIENT_NOT_FOUND	No patient profile for the user
404	APPOINTMENT_NOT_FOUND	No appointment with the given ID
404	OVERRIDE_NOT_FOUND	No active temporary override
409	EMAIL_EXISTS	Email already registered
409	APPOINTMENT_OVERLAP	Time slot overlaps existing appointment
409	OVERRIDE_EXISTS	Doctor already has a temp override
409	ALREADY_CANCELLED	Appointment was already cancelled
422	TOO_SOON	Appointment less than 24h in the future
422	CANCELLATION_TOO_LATE	Less than 12h before appointment
422	OUTSIDE_WORKING_HOURS	Appointment outside effective hours
422	INVALID_TIME_RANGE	End before start, or cross-day
422	EFFECTIVE_DATE_TOO_SOON	Permanent change < 7 days ahead
422	SCHEDULE_CONFLICTS_APPOINTMENT	Schedule change conflicts with bookings
422	INVALID_OVERRIDE_WINDOW	Override end before start

HTTP Status Code Summary

Code	Meaning	Used For
200	OK	Successful GET, PUT
201	Created	Successful POST that creates a resource
204	No Content	Successful DELETE
400	Bad Request	Malformed JSON / missing fields (Pydantic)
401	Unauthorized	Missing or invalid JWT
403	Forbidden	Wrong role or not the resource owner
404	Not Found	Resource does not exist

409	Conflict	Duplicate / overlap / already exists
422	Unprocessable Entity	Business rule violation

2. Architecture Description

2.1 High-Level Architecture

The application follows a layered architecture with clear separation between HTTP concerns, business logic, and data persistence:

```
+-----+
| HTTP Client |
+-----+
      | JSON over HTTP
+-----V-----+
| FastAPI Routers | Thin HTTP adapters: parse request,
| (auth, doctors, | call service, return response.
| patients,       | No business logic here.
| schedules,      |
| appointments)   |
+-----+
      |
+-----V-----+
| Service Layer   | All business rules, validations,
| (**/service.py) | and orchestration live here.
+-----+         Raises custom AppException subclasses.
      |
+-----V-----+
| SQLAlchemy ORM  | Declarative models define tables,
| (**/models.py) | relationships, and constraints.
+-----+
      |
+-----V-----+
| SQLite (async)  | aiosqlite driver; Alembic migrations
+-----+         manage schema evolution.
```

Cross-cutting:

- Pydantic schemas (**/schemas.py) define API contracts
- Custom exceptions (common/exceptions.py)
- Global error handlers (common/error_handlers.py)
- JWT middleware (dependencies.py + auth/security.py)

2.2 Project Structure

The codebase uses feature-based module organization. Each domain concept (auth, doctors, patients, schedules, appointments) has its own package with router, service, schemas, and models. Shared utilities live in common/.

```
app/
+-- main.py          App factory, lifespan, router mounting
+-- config.py        Pydantic-settings (.env loading)
+-- database.py       Async engine, session factory, Base
+-- dependencies.py   get_db, get_current_user, role guards
+-- auth/
| +-- router.py       POST register/doctor, register/patient, login
| +-- service.py       Registration, login logic
| +-- schemas.py       Request/response Pydantic models
| +-- security.py      JWT create/decode, password hash/verify
| +-- models.py        User model (shared auth identity)
```

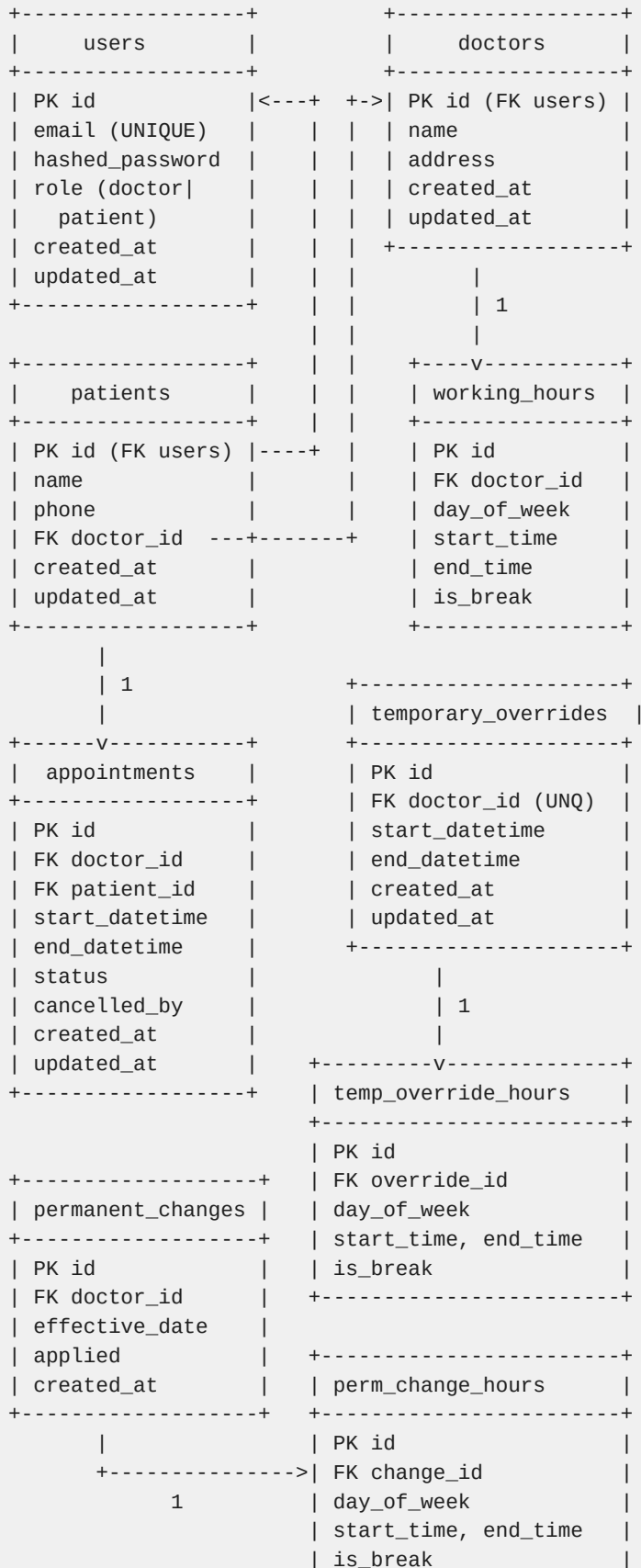
```

+-- doctors/
| +-- router.py      GET /doctors, GET /doctors/{id}
| +-- service.py    List + detail with eager-loaded hours
| +-- schemas.py    DoctorResponse, DoctorListResponse
| +-- models.py     Doctor model (1:1 FK to users)
+-- patients/
| +-- router.py      GET /patients/me
| +-- service.py    Profile retrieval
| +-- schemas.py    PatientResponse
| +-- models.py     Patient model (FK to users + doctors)
+-- schedules/
| +-- router.py      Schedule CRUD endpoints
| +-- service.py    CRUD + effective schedule resolution
| +-- schemas.py    TimeSlot, Override, PermanentChange DTOs
| +-- models.py     WorkingHours, TempOverride, PermanentChange
| +-- slot_fitting.py Shared interval-in-slot predicate
| +-- appointment_conflicts.py Pre-commit conflict check
+-- appointments/
| +-- router.py      POST, DELETE, GET /appointments
| +-- service.py    Create, cancel, list + validations
| +-- schemas.py    AppointmentCreate/Response/ListFilters
| +-- models.py     Appointment model
+-- common/
| +-- exceptions.py  AppException hierarchy
| +-- error_handlers.py Global exception -> JSON handlers

```

2.3 Database Schema (ER Diagram)

Seven tables with referential integrity enforced via foreign keys. SQLite PRAGMA foreign_keys=ON is set on every connection.



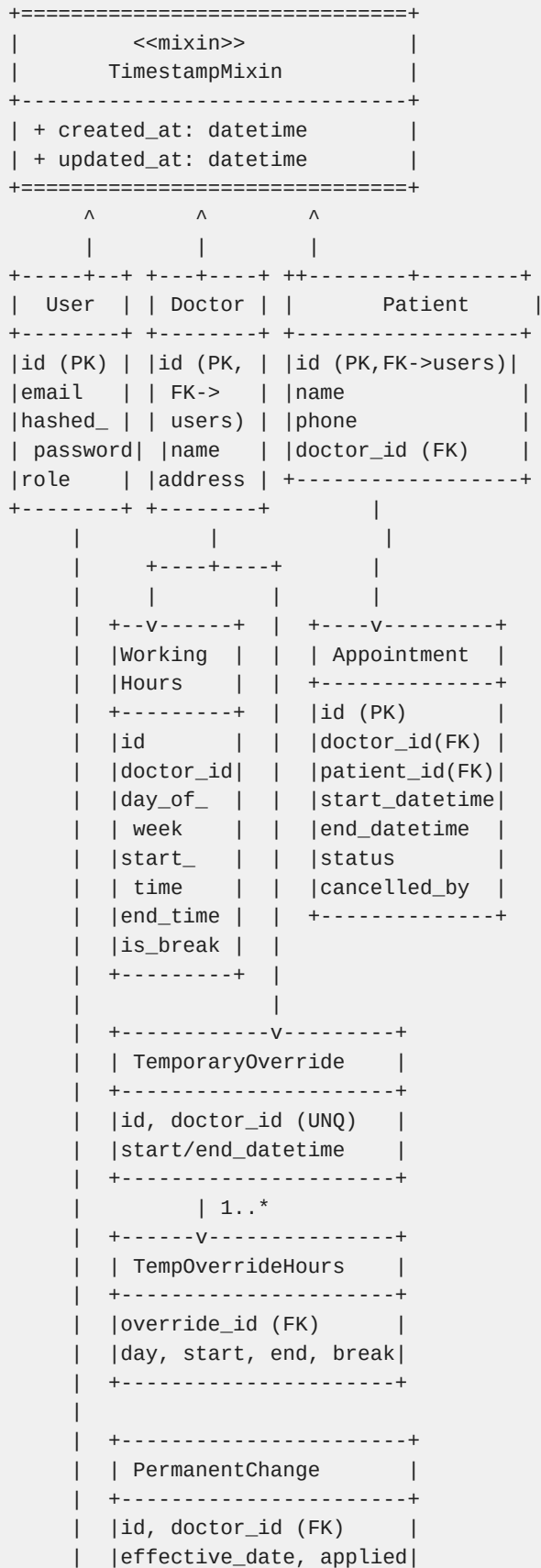
+-----+

Key constraints:

- users.email has a UNIQUE index
- temporary_overrides.doctor_id is UNIQUE (max 1 per doctor)
- working_hours has UNIQUE(doctor_id, day_of_week, start_time)
- Appointments have a composite index on (doctor_id, start_datetime, end_datetime)
- CHECK constraints enforce valid role, status, and day_of_week values

2.4 Class Diagram (UML)

The class diagram below shows the main domain models and their relationships. All models inherit from SQLAlchemy's DeclarativeBase. Models with timestamps mix in TimestampMixin.




```

| +-----+
| | 1..* |
| +-----v-----+
| | PermanentChangeHours |
| +-----+
| | change_id (FK) |
| | day, start, end, break |
| +-----+

```

Service Layer (key classes / functions):

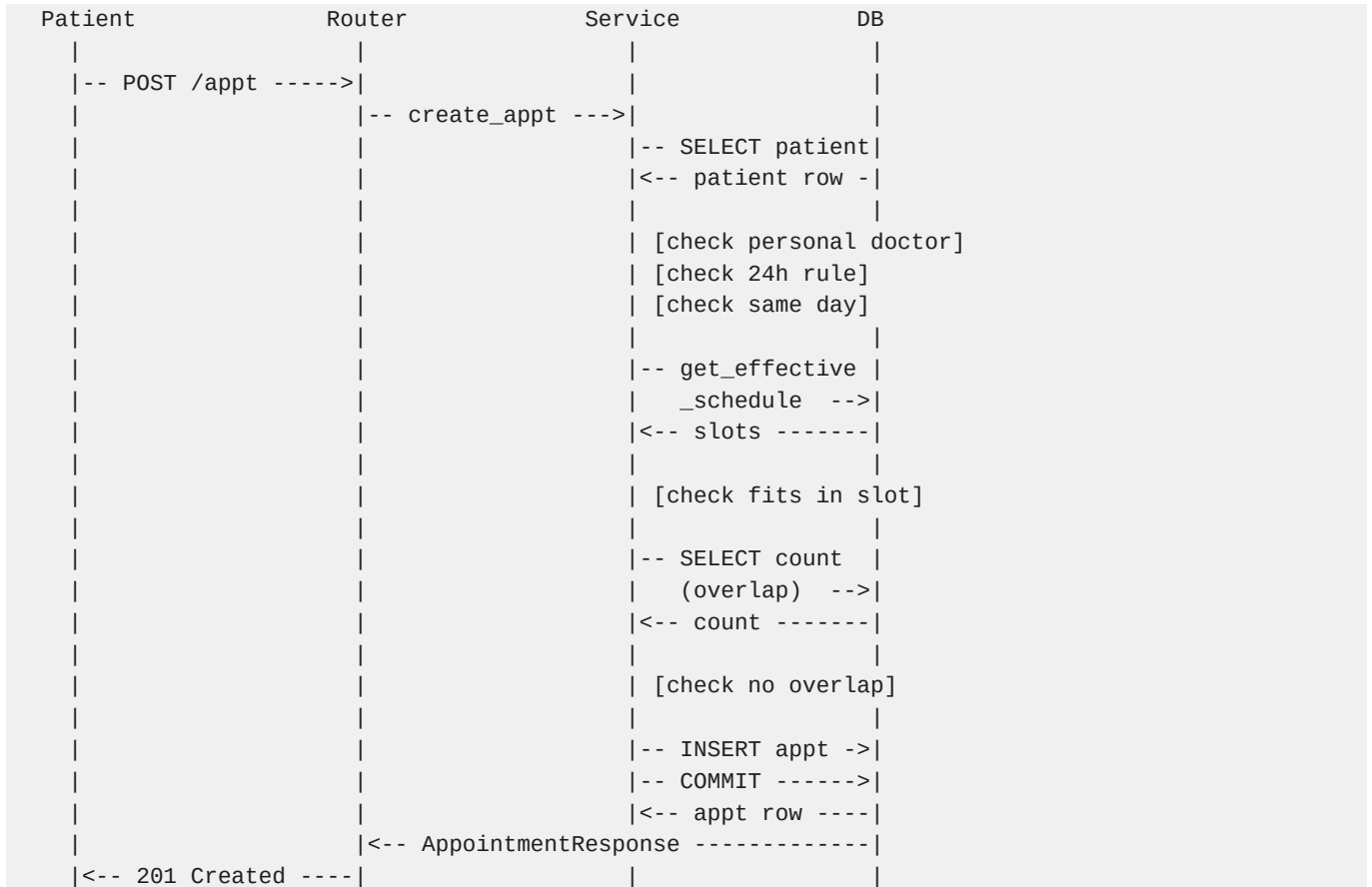
```

+-----+
| auth/service.py |
| register_doctor(), register_patient(), |
| authenticate() |
+-----+
| schedules/service.py |
| get_effective_schedule() |
| update_working_hours() |
| create_temporary_override() |
| create_permanent_change() |
| apply_pending_permanent_changes() |
+-----+
| appointments/service.py |
| create_appointment() |
| cancel_appointment() |
| list_appointments() |
+-----+
| dependencies.py |
| get_db(), get_current_user() |
| require_doctor(), require_patient() |
+-----+

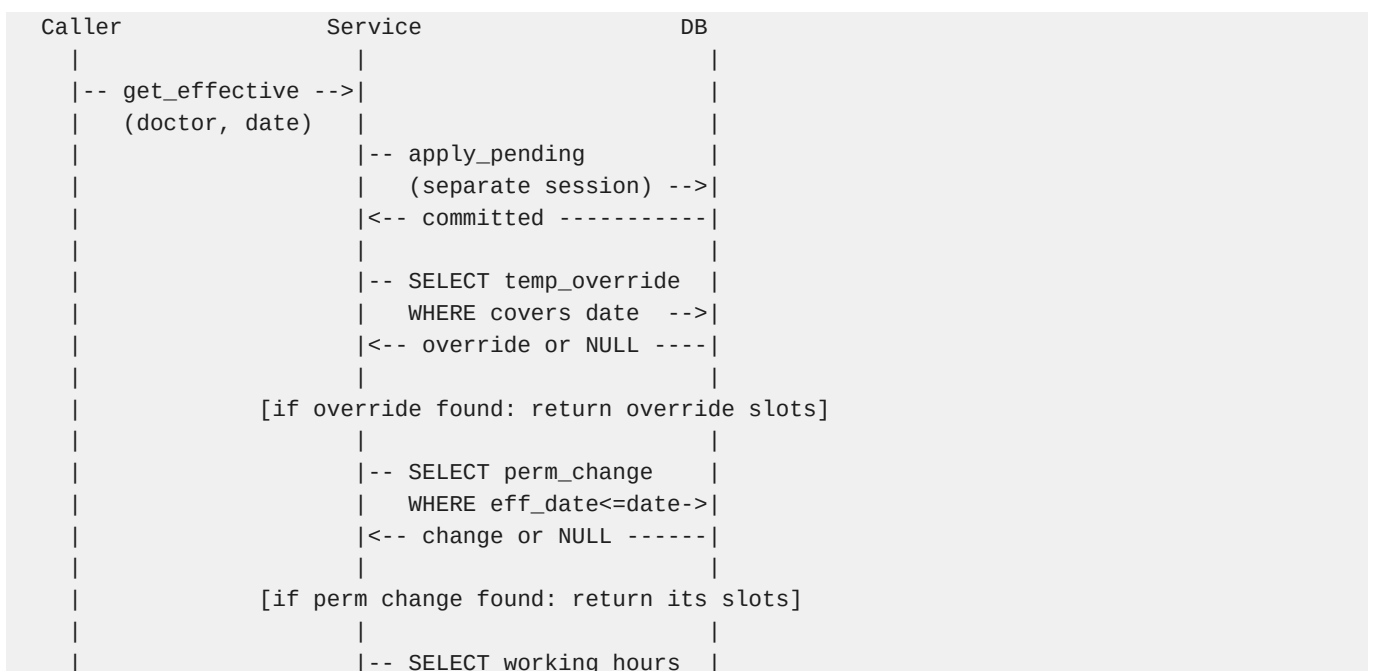
```

2.5 Sequence Diagrams

2.5.1 Create Appointment Flow



2.5.2 Schedule Resolution (get_effective_schedule)



```
|
|      WHERE weekday ----->|
|      <-- base slots -----|
|
|<-- list[TimeSlot] -|
```

2.5.3 Schedule Update with Conflict Check

Doctor	Router	Service	DB
-- PUT schedule --->			
	-- update_wh ----->		
		-- DELETE old -->	
		-- INSERT new -->	
		-- FLUSH ----->	
		-- assert_no	
		conflicts --->	
		(for each	
		scheduled	
		appointment:	
		check fits	
		new slots)	
	[if conflict: ROLLBACK, raise 422]		
	[if ok: COMMIT]		
<-- 200 OK -----			

3. Self-Analysis

3.1 SOLID Principles Adherence

Single Responsibility Principle (SRP)

Each module handles exactly one domain concept. Within a module, responsibilities are further separated:

- Routers handle HTTP parsing and response formatting only.
- Services contain all business logic, validation, and orchestration.
- Models define persistence structure and database constraints.
- Schemas define the API contract (request validation, response shape).
- The common/ package centralizes cross-cutting concerns: exception classes in exceptions.py and global error handlers in error_handlers.py.

Example: appointments/service.py owns all booking rules (24h, overlap, working hours). The router never checks business rules; it delegates entirely to the service.

Open/Closed Principle (OCP)

The schedule resolution system demonstrates OCP. The `get_effective_schedule()` function checks three schedule layers in priority order (temporary override > permanent change > base working hours). Adding a new layer (e.g., holiday calendars) requires adding a new model and inserting one more check step, without modifying the existing override or permanent-change logic.

The exception hierarchy (`AppException` -> `BusinessRuleException`, `ConflictException`, etc.) is also open for extension. New error types can be added without changing the global error handler, which catches the base `AppException` class.

Liskov Substitution Principle (LSP)

Both Doctor and Patient link to User via a 1:1 foreign key. Any code that operates on a User (authentication, token creation, `get_current_user` dependency) works identically regardless of whether the user is a doctor or patient. The role field is used for authorization checks (`require_doctor`, `require_patient`), but the authentication flow itself is fully substitutable.

Interface Segregation Principle (ISP)

Pydantic schemas are scoped per operation. `DoctorRegisterRequest`, `PatientRegisterRequest`, `LoginRequest`, and `TokenResponse` are all separate models. The appointment module has distinct `AppointmentCreateRequest`, `AppointmentResponse`, and `AppointmentListFilters`. No client is forced to deal with fields irrelevant to their operation.

FastAPI dependencies (`get_current_user`, `require_doctor`, `require_patient`) are fine-grained. Endpoints declare exactly the auth level they need.

Dependency Inversion Principle (DIP)

Services depend on the `AsyncSession` abstraction, never on concrete engine details. This is demonstrated by the

test suite, which seamlessly swaps the production SQLite file for an in-memory database by overriding the `get_db` dependency. No service code changes are needed.

The dependency injection pattern via FastAPI's `Depends()` mechanism decouples request handling from database session management and authentication.

3.2 Design Trade-offs

SQLite vs. PostgreSQL

SQLite was chosen for simplicity (zero-config, file-based). This suits the project scope perfectly but limits concurrent write throughput and advanced features like LISTEN/NOTIFY. The async driver (`aiosqlite`) wraps synchronous SQLite in a thread pool. Migration to PostgreSQL requires only changing `DATABASE_URL` and installing `asyncpg`; all queries use the ORM and are database-agnostic.

Eager Promotion of Permanent Changes

Permanent schedule changes are promoted to base `working_hours` both at startup (lifespan) and on-demand before each `get_effective_schedule()` call. This is simple and correct but performs a global scan of all unapplied changes on every schedule read. For a course-scale application this is negligible; at scale, a background scheduler (e.g., Celery beat, APScheduler, or a cron job) would be more appropriate.

Appointment Conflict Check on Schedule Mutation

When a doctor updates their schedule, the system checks all scheduled appointments against the new effective hours and rejects the change if any appointment would be orphaned. An alternative design would flag or auto-cancel conflicting appointments, but the reject-with-422 approach was chosen for safety: no patient loses a booking without explicit action.

JWT Without Refresh Tokens

The current auth system issues a single access token with a 60-minute expiry. There is no refresh token flow. This simplifies the implementation but means clients must re-authenticate after expiry. Adding refresh tokens would be a natural enhancement for production use.

3.3 Areas for Improvement

- Pagination consistency: Use cursor-based pagination instead of offset/limit for better performance on large datasets.
- Rate limiting: No rate limiting is implemented. Production deployments should add middleware or reverse-proxy-level throttling.
- Refresh tokens: Implement a token refresh flow to improve UX without compromising security.
- Email notifications: Notify patients and doctors on appointment creation, cancellation, and schedule changes.
- Admin endpoints: No admin role exists. Adding one would enable centralized management of users, appointments, and schedules.
- Soft-delete and audit trail: Cancelled appointments set a status flag, but other entities use hard deletes. A comprehensive audit log would improve traceability.
- Timezone handling: All datetimes are stored and processed in UTC. Client-side timezone conversion is left

to the consumer. An explicit timezone-aware API (accepting IANA tz names) would improve usability.

- Background job for promotions: Replace the on-demand `apply_pending_permanent_changes` with a dedicated scheduler to avoid per-request overhead at scale.
- Docker health checks: The Dockerfile and `docker-compose.yml` do not include a HEALTHCHECK directive. Adding one would improve container orchestration.
- API versioning: All routes are currently unversioned (`/auth`, `/doctors`, etc.). Prefixing with `/v1/` would enable backward-compatible evolution.